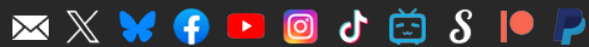
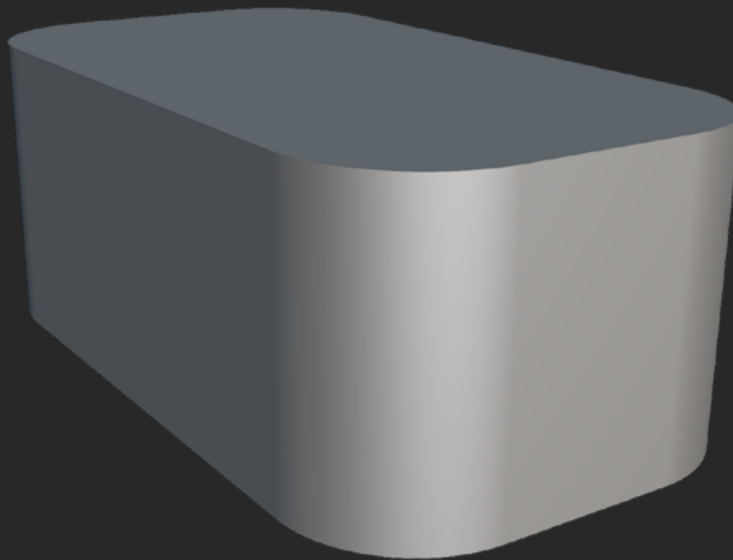


Inigo Quilez :: articles :: rounded boxes - 2024

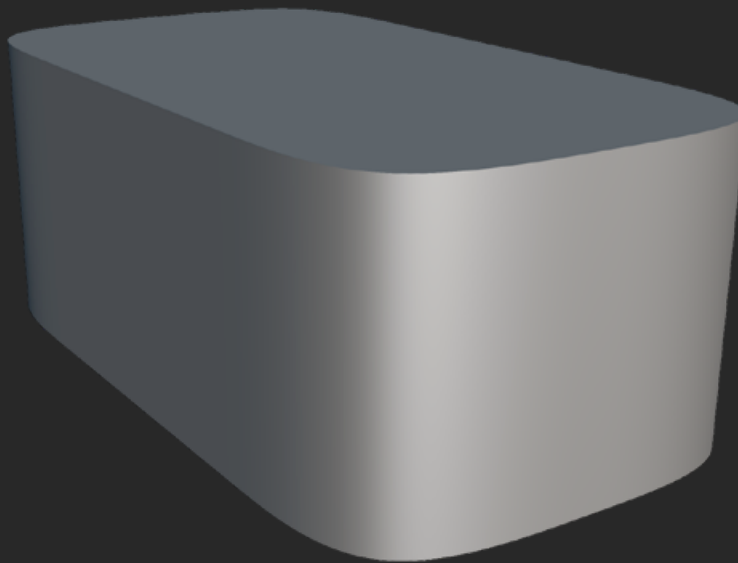


Intro

We all know that if you chop the edges of a box and round them with a circular chamfer, then the surface you get has second degree discontinuities. That means that the surface normals, which are the first derivatives of the surface, are not smooth. And if the normals are not smooth, then so won't be the lighting either. Indeed, a box with cylindrical edges almost was funny ugly lighting on it.



Traditional rounded box with circular corners



Rounded box with cubic corners

It is also well known that you can also achieve "rounded box" shapes with other method, such as using generalized ellipses like $f(x,y) = (x/a)^n + (y/b)^n - 1$ or others. However those approaches produce pretty awkward looking shapes, without any parallel sides, which can be a problem depending on the application, and aesthetically pleasant depending on your taste.

So, instead on giving up on rounded boxes, let's work a bit harder and see what we can do about the problem.

The setup

So, let's consider the top right corner of a 2D box or rectangle. We are going to chop it from the rectangle, and replace it with some shape or curve, such that where we normally would attach an arc of a circle, we'll now attach something else. Because the derivatives are important for continuity, and because on the part where this curve connect with the right side it needs to come down perfectly vertical and that creates an infinite derivative which is a bit of a pain to handle, we are going to make our lives easier by rotating the whole setup by 45 degrees, without loss of generality. We are also going to put the origin of coordinates at the corner of the chopped rectangle, and scale the whole setup so that the side length of the chopped rectangle is exactly 1. We can call this the "uv" coordinate system. You can see the coordinate system in the figure after the next paragraph.

Here you can see the traditional circular corner and its associated corner function $c(u)=\sqrt{(2-u^2)-1}$, which is a circle of radius $\sqrt{2}$ centered at (0,-1).

The code below computes the SDF of our rounded box by performing this coordinate transformation, and then invoking the SDF of the corner shape, `corner_sdf()`, which in this case is the SDF of our circle as just described:

```
float sdf_corner_circle( vec2 uv )
{
    return length(uv-vec2(0.0,-1.0))-s
}

float sdRoundedBox( float p, float b,
{
    // select corner radius
    r.xy = (p.x>0.0)?r.xy:r.zw;
    r.x = (p.y>0.0)?r.x :r.y;

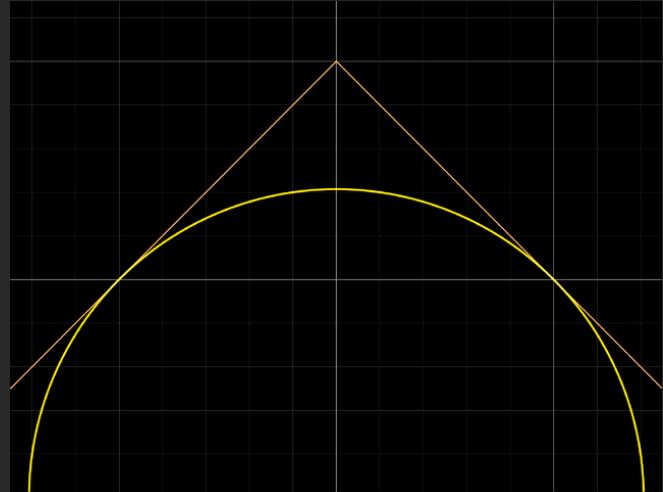
    // box coordinates
    vec2 q = abs(p)-b+r.x;

    // distance to sides
    if( min(q.x,q.y)<0.0 ) return max(

    // rotate, mirror, offset and scal
    // to UV corner coordinates
    vec2 uv = vec2(abs(q.x-q.y),q.x+q.

    // distance to corner
    float d = sdf_corner_circle( uv );

    // undo scale and return
    return d * sqrt(2.0)*r.x;
}
```



Corner of the rounded box, in the uv space

If you expand all the code above and cancel out terms and do all the simplifications, you'll arrive to the traditional and optimized formulation for the SDFs of a rounded box.

Now all we need to do is use different corner shape SDFs and we'll be able to create a multitude of alternatives to the rounded box, and maybe solve the continuity problems. But for that let's look quickly to why exactly the circle does not actually satisfy the required continuity constraints:

In the uv corner coordinate system, the side of our box is the function $c(u) = 1-u$ so its derivatives are

	$c(u)$	$c'(u)$	$c''(u)$
Straight	$1-u$	-1	0

Great. Now let's look at the circle we use traditionally, which is again a circle centered at $(0,-1)$ and radius $\sqrt{2}$, so $c(u) = \sqrt{(2-u^2)}-1$. Its derivatives are

	$c(u)$	$c'(u)$	$c''(u)$
Circular	$\sqrt{(2-u^2)}-1$	$-u/\sqrt{(2-u^2)}$	$-2(2-u^2)^{-3/2}$

So if we now compare both straight sides and circular corner and pay attention to the value of the second derivatives of these two pieces at the point where the corner meets the sides, which is the point $(1,0)$, we get:

	$c(u)$	$c'(u)$	$c''(u)$	$c(1)$	$c'(1)$	$c''(1)$
Straight	$1-u$	-1	0	0	-1	0
Circular	$\sqrt{(2-u^2)}-1$	$-u/\sqrt{(2-u^2)}$	$-2(2-u^2)^{-3/2}$	0	-1	-2

And in the last column lays the problem: while the surfaces touch (function values coincide) and are tangent to each other perfectly (derivatives match too), the second derivative or "motion" of the normals doesn't - in the case of the box sides the normals do not want to change along the sides, but the circular arc wants to keep the normals moving. This mismatch in the normal regime is what makes the lighting respond non-smoothly.

Exploring alternative #1 - parabolas and hyperbolas

Shadertoy user wutzhuhn proposed exploring a parabola instead of circles hoping that would resolve the problem, so let's follow his idea. The parabola that passes through the connection point $(1,0)$ and has slope -1 at that point is $c(u) = (1-u^2)/2$. We can then double check that the first and second derivatives and evaluate the second derivative see if we are lucky and get a zero (no need to check function value and first derivative since those are set by construction)

	$c(u)$	$c'(u)$	$c''(u)$	$c''(1)$
Straight	$1-u$	-1	0	0

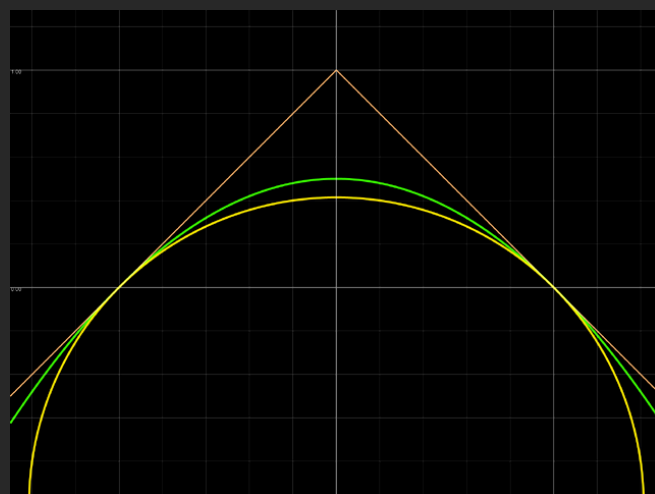
Circular	$\sqrt{(2-u^2)}-1$	$-u/\sqrt{(2-u^2)}$	$-2(2-u^2)^{-3/2}$	-2
Parabola	$(1-u^2)/2$	-u	-1	-1

Well, unfortunately the second derivative is still not zero, hence the parabolic corner also does not smoothly connect to the sides of the box. On the bright side, -1 is closer to 0 than -2, so there is a smaller disparity in the case of parabolas that there was with circles, which can help a little hide the ugly lighting situation. On the other hand, in the context of SDFs rendering, computing the exact SDF of a parabola is doable (see [this link](#)) but it's very expensive compared to the circle and probably not worth the effort.

Here goes the code for the parabolic corner that you can use in the code above to replace the old `sdf_corner_circle()`:

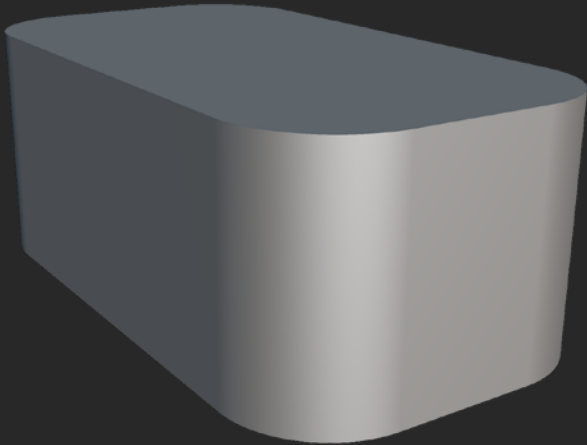
```
float sdf_corner_parabola( vec2 uv )
{
    float y = (0.5 + uv.y)*(2.0/3.0);
    float h = uv.x*uv.x + y*y*y;
    float w = pow( uv.x + sqrt(abs(h))
    float x = w - y/w;
    vec2 q = vec2(x, 0.5*(1.0-x*x));
    return length(p-q)*sign(uv.y-q.y);
}
```

Please note that I have taken the liberty of optimizing a bit the SDF of the parabola by ignoring the case where three roots solve the involved cubic equation. I did this because the regions of space where this can happen are minuscule and only happen deep inside the box.

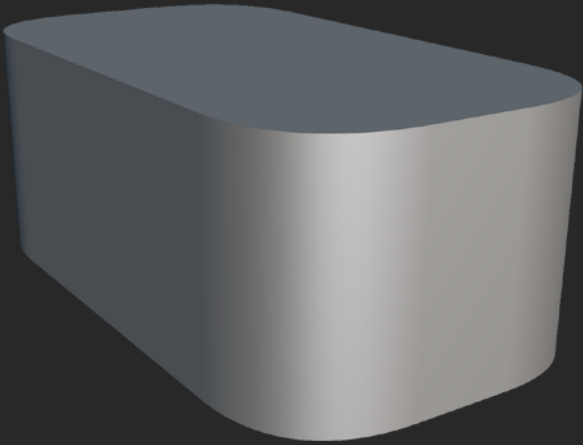


Circle and Parabola, in uv space

As explained, the parabolic corner shapes does NOT fix the lighting discontinuity problem; but it does make it half as pronounced indeed:



Traditional Circle rounded box



New Parabolic rounded box

Now, besides the circle and the parabola, there's another conic section that we can use to construct the corner - the hyperbola. In this case, we have $c(u) = (k+1) - \sqrt{(k+1)} \cdot \sqrt{(k+u^2)}$, which gives us the following derivatives:

	$c(u)$	$c'(u)$	$c''(u)$
Straight	$1-u$	-1	0
Circular	$\sqrt{(2-u^2)}-1$	$-u/\sqrt{(2-u^2)}$	$-2(2-u^2)^{-3/2}$
Parabola	$(1-u^2)/2$	$-u$	-1
Hyperbola	$(k+1)-\sqrt{(k+1)} \cdot \sqrt{(k+u^2)}$	$-\sqrt{(k+1)} \cdot u/\sqrt{(k+u^2)}$	$-k\sqrt{(k+1)}/\sqrt{(k+u^2)^3}$



Circle and Parabolic, Hyperbola, in uv space

As you can see, the second derivative is not zero, unless we make $k=0$, in which case the hyperbola becomes two straight lines, not a curved shape. On the other hand it's nice to have an extra free parameter that you can play with and find the right tradeoff between lighting smoothness and box roundness.

Getting ready to explore more curves

Before we try exploring more curves, we should probably find a method to compute the SDF of any particular curve that we want, beyond circles, parabolas and hyperbolas. In particular, given any 2D curve $c(t)$, we want to compute its SDF $f(u,v)$. What we can do is look at all points in the curve parametrized by t , measure their distance to the target point uv , and pick the one that produces the smallest distance squared (we square distance to save ourselves a square root). Basically, if we call our SDF $f(u,v)$, then we want the t for which

$$f^2(u,v) = (u-t)^2 + (v-c(t))^2$$

is minimum. That is, we are looking for $\partial f^2 / \partial t = 0$, which simplifies to

$$(u-t) + [v-c(t)] \cdot c'(t) = 0$$

We now solve for t , plug in first equation, take a square root, and that is our SDF formula.

If you do this as an exercise for the circular profile $c(t) = \sqrt{2-t^2}-1$, you will find, after quite some work simplifying and rearranging terms, that you get indeed the SDF $f(u,v) = \sqrt{[u^2+(v+1)^2]} - \sqrt{2}$ which is the SDF of a circle centered at $(0,-1)$ and radius $\sqrt{2}$ indeed.

All right, we are ready to continue with our second experiment for our box corners:

Exploring alternative #2 - Trigonometrics

Since we clearly need our second derivatives to be forced, one alternative is to use a cosine function instead of circles or parabolas. For example,

$$c(u) = 2/\pi \cdot \cos(u\pi/2)$$

which I've scaled and offset to connect perfectly with the straight sides of the box. Let's analyze how it behaves:

	$c(u)$	$c'(u)$	$c''(u)$	$c''(1)$
Straight	$1-u$	-1	0	0
Circular	$\sqrt{(2-u^2)}-1$	$-u/\sqrt{(2-u^2)}$	$-2(2-u^2)^{-3/2}$	-2
Parabola	$(1-u^2)/2$	$-u$	-1	-1
Cosine	$2/\pi \cdot \cos(u\pi/2)$	$-\sin(u\pi/2)$	$-\pi/2 \cdot \cos(u\pi/2)$	0

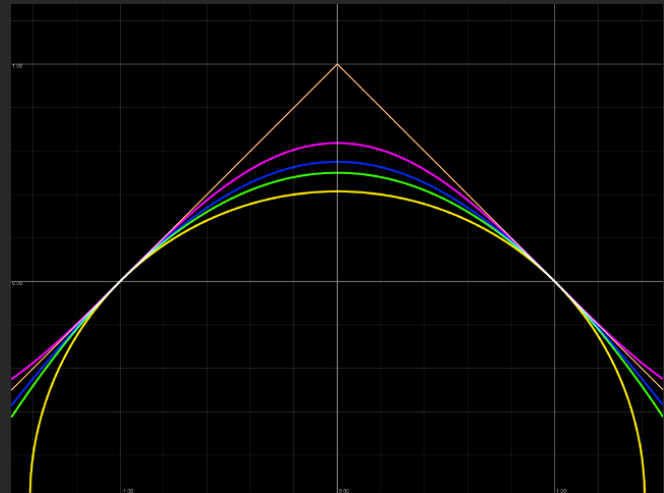
Finally, this is great news, our cosine should solve our lighting problem. The problem we do have now is that if we replace $c(u)$ and $c'(u)$ above into our SDF computation we described previously, then we get some pretty nasty trigonometric equation:

$$(u-t) + [v-c(t)] \cdot c'(t) = 0$$

So while there is no closed form solution that we can write, we can at least implement a basic binary search:

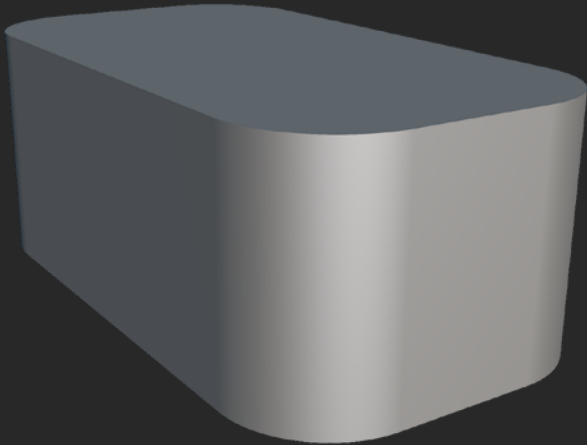
```
float sdf_corner_cosine( vec2 uv )
{
    const float kPI = 3.1415927;
    uv *= (kPI/2.0);

    float ta = 0.0, tb = kPI/2.0;
    for( int i=0; i<12; i++ )
    {
        float t = 0.5*(ta+tb);
        float y = t-uv.x+(uv.y-cos(t))
        if( y<0.0 ) ta = t; else tb =
    }
    vec2 qa = vec2(ta,cos(ta));
    vec2 qb = vec2(tb,cos(tb));
    vec2 pa = uv-qa, di = qb-qa;
    float h = clamp(dot(pa,di)/dot(di,
    return (2.0/kPI) * length(pa-di*h)
        sign(pa.y*di.x-pa.x*di.y);
}
```

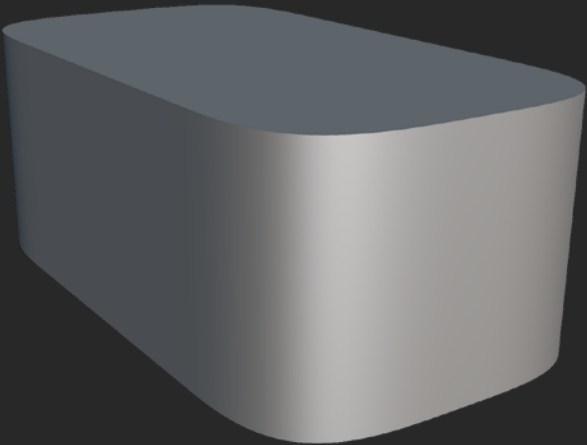


Circle, Parabola, Hyperbola, Cosine

And indeed, the cosine shaped corner DOES finally produce smooth lighting on our rounded box:



Traditional Circle rounded box



New Cosine rounded box

Exploring alternative #3 - Cubic polynomial

We might also try to find other curves without resorting to trigonometrics. Of course, circles, parabolas and hyperbolas are conic sections, or second degree curves. All it really takes to constrain the second derivative to zero is to raise the degree of the polynomial corner to a third degree, it's that simple. So, let's take the generic cubic polynomial

$$c(u) = a_3u^3 + a_2u^2 + a_1u + a_0$$

and add the constraints $c(1)=0$, $c'(1)=-1$, $c''(1)=0$ and $c'(0)=0$, then we can solve a little linear system of equations and get $a_3=1/3$, $a_2=-1$, $a_1=0$, $a_0=2/3$, or in other words, we get

$$c(u) = (u^3 - 3u^2 + 2)/3$$

which has roots $u = 1$, and $u = 1 \pm \sqrt{3}$. Our compelte table of derivative comparisons is now:

	$c(u)$	$c'(u)$	$c''(u)$	$c''(1)$
Straight	$1-u$	-1	0	0
Circular	$\sqrt{(2-u^2)}-1$	$-u/\sqrt{(2-u^2)}$	$-2(2-u^2)^{-3/2}$	-2
Parabola	$(1-u^2)/2$	$-u$	-1	-1

Cosine	$2/\pi \cdot \cos(u\pi/2)$	$-\sin(u\pi/2)$	$-\pi/2 \cdot \cos(u\pi/2)$	0
Cubic	$(u^3 - 3u^2 + 2)/3$	$u^2 - u \cdot 2$	$2u - 2$	0

Alright, now we need to compute the distance to this cubic equation! If we apply the curve to the SDF method outlined earlier to our cubic curve, we get that since $c(u)$ is of degree 3, $c(u)'$ is of degree 2. So that means that our minimization function

$$(u-t) + [v-c(t)] \cdot c'(t) = 0$$

will be of degree 5, since c multiplies c' . This is bad news again, we cannot solve general quintic equations in close form with the instructions we have in the GPUs today (square roots, logs and trigonometrics).

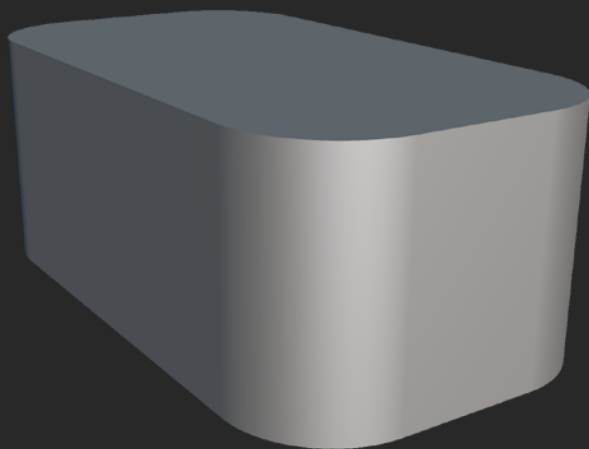
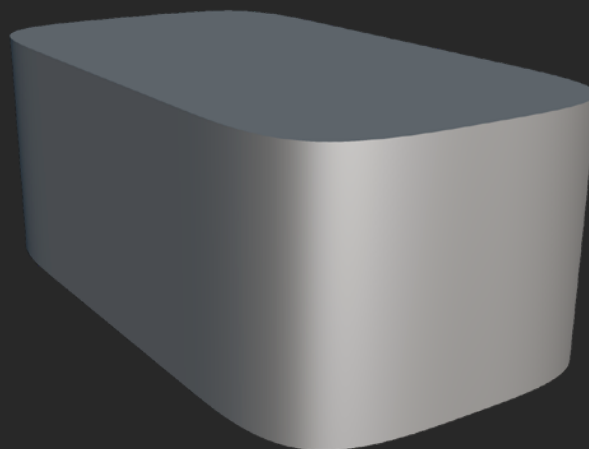
So we need to resort to a numerical solver again, and just like with the cosine, we'll use a binary search to find the root:

```
float sdf_corner_cubic( vec2 uv )
{
    float ta = 0.0, tb = 1.0;
    for( int i=0; i<12; i++ )
    {
        float t = 0.5*(ta+tb);
        float c = (t*t*(t-3.0)+2.0)/3.0;
        float dc = t*(t-2.0);
        float y = (uv.x-t) + (uv.y-c)*dc;
        if( y>0.0 ) ta = t; else tb = t;
    }
    vec2 qa = vec2(ta, (ta*ta*(ta-3.0)+2.0)/3.0);
    vec2 qb = vec2(tb, (tb*tb*(tb-3.0)+2.0)/3.0);
    vec2 pa = uv-qa, di = qb-qa;
    float h = clamp( dot(pa,di)/dot(di,di), 0.0, 1.0 );
    return length(pa-di*h) * sign(pa.y);
}
```



Circle, Parabola, Hyperbola, Cosine, Cubic

As expected, the cubic corner also produces a smooth rounded box, although is slightly cheaper to evaluate than the cosine one:

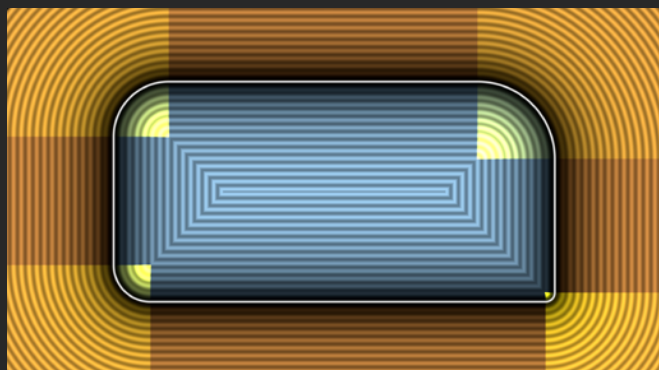
Traditional **Circle** rounded boxNew **Cubic** rounded box

You can see a visual comparison of all these alternatives in this Shadertoy shader here:

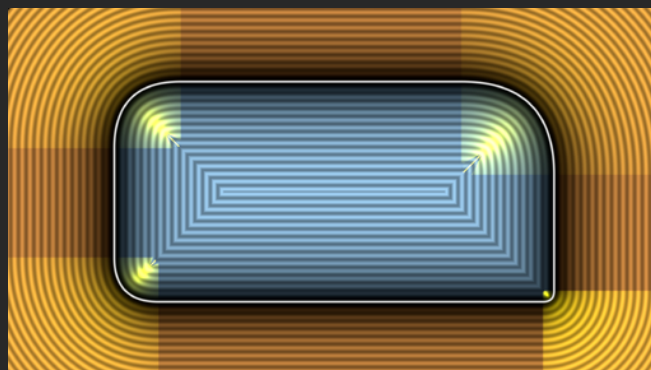
<https://www.shadertoy.com/view/4fyGz1>, where you'll also find the source code:

Visualizing the derivatives

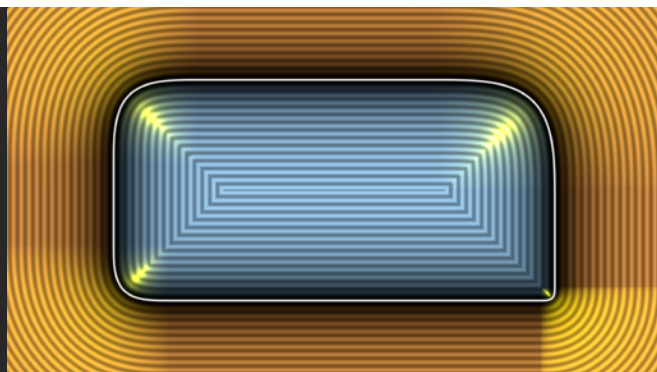
It's never a bad idea to visualize some of the quantities of any given problem to gain some deeper intuition as to how things are happening. Here goes a display of the second derivatives of the SDF shape in yellow color. In the pictures below I'm showing the SDF and the Laplacian of the SDF in yellow. The Laplacian $\mathcal{L} = \partial^2 \text{sdf} / \partial x^2 + \partial^2 \text{sdf} / \partial y^2$ is made of the second derivatives of the SDF, so discontinuities in 3D lighting will show as discontinuities in the yellow colors too:



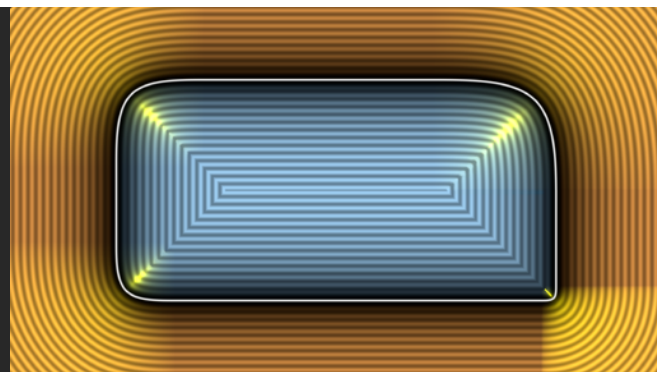
Circular



Parabolic



Cosine



Cubic

Indeed you can see how the Cosine and Cubic variants have a smooth yellow gradient over the plane as opposed to the Circular and the Parabolic. You can find the code producing these images and a real time animation here: <https://www.shadertoy.com/view/4cG3R1>.

Normalization

The way I coded the rounded box function above, the main parameter to control the rounding is the size of its "radius" in the case of the circular corner, or more generally, the size of the rectangles that we are chopping off the box. From there, each one of the different corner shapes we've explored does its best to fit in it smoothly, resulting in a different maximum amplitude or "size" for the corner. You can see that in the 2D graphs of the different curves $c(u)$ that we designed.

But if we wanted to normalize all the corner shapes so the boxes would stay similar regardless of the corner shape in use, we could divide the main corner radius parameter by the maximum of each of the curves $c(u)$, which happens at $c(0)$. These are the normalization values:

	$c(0)$
Circular	$\sqrt{2}-1$
Parabola	$1/2$
Cosine	$2/\pi$
Cubic	$2/3$

inigo quilez - learning computer graphics since 1994